# Measurement and strategy



## 8C. Measurement & Strategy

**Questions:** 6 **Difficulty:** Mid to Senior **Companies:** Google, Amazon, Flipkart, Razorpay, Swiggy, Microsoft

---

### Q 8.13 When do you use Gzip vs Brotli? How does compression work?

**Difficulty:** Mid **Asked at:** Google, Amazon, Flipkart

Text-based resources (HTML, CSS, JS, JSON, SVG) compress dramatically — typically 60-80% smaller. A 200KB JavaScript bundle might compress to 50KB over the wire.

**Brotli vs Gzip:**

Brotli compresses 15-25% better than Gzip at equivalent speed. All modern browsers support it (`Accept-Encoding: br`). Most CDNs (Cloudflare, Vercel, AWS CloudFront) serve Brotli by default.

Gzip is the fallback for older clients and is still universally supported. If your server only supports one, Gzip is fine. If it supports both, serve Brotli to browsers that accept it and Gzip to the rest.

**Static vs dynamic compression:**

Static compression: compress files at build time, serve the pre-compressed version. Best for static assets (JS, CSS) that don't change per request. Maximum compression

BASH Copy

```
# Build step: pre-compress all static assets
brotli -q 11 dist/app.js -o dist/app.js.br
gzip -9 dist/app.js -o dist/app.js.gz
```

Dynamic compression: the server compresses the response on-the-fly per request. Necessary for dynamic content (HTML, API responses). Uses lower compression levels for speed.

**What NOT to compress:** Images (JPEG, PNG, WebP, AVIF are already compressed), videos, fonts (woff2 is already compressed), and any binary format that's already compressed. Compressing these wastes CPU and might even increase file size.

**Follow-up:** "How do you verify compression is working?"

Check the response headers in DevTools → Network tab. Look for `Content-Encoding: br` (Brotli) or `Content-Encoding: gzip`. The "Size" column shows both transferred size (compressed) and actual size (decompressed).

---

## Q 8.14 How do you optimize API calls from the frontend?

**Difficulty:** Mid **Asked at:** Razorpay, Flipkart, Swiggy, Amazon

**Request batching — combine multiple calls into one:**

JAVASCRIPT Copy

```
// BAD: 3 separate API calls
const user = await fetch('/api/user/123');
const orders = await fetch('/api/user/123/orders');
const notifications = await fetch('/api/user/123/notifications');

// BETTER: Parallel calls (still 3 requests, but not sequential)
const [user, orders, notifications] = await Promise.all([
  fetch('/api/user/123'),
```

```
// BEST: Batch endpoint (1 request)
const data = await fetch('/api/batch', {
  method: 'POST',
  body: JSON.stringify({
    requests: [
      { url: '/user/123' },
      { url: '/user/123/orders' },
      { url: '/user/123/notifications' },
    ],
  }),
});
```

`AbortController` — cancel stale requests:

JAVASCRIPT Copy

```
function useSearch(query) {
  const [results, setResults] = useState([]);
  const controllerRef = useRef(null);

  useEffect(() => {
    // Cancel the previous request if the user types again
    controllerRef.current?.abort();
    controllerRef.current = new AbortController();

    fetch(`/api/search?q=${query}`, { signal: controllerRef.current.signal
      .then(r => r.json())
      .then(setResults)
      .catch(err => {
        if (err.name !== 'AbortError') throw err;
      });

    return () => controllerRef.current?.abort();
  }, [query]);

  return results;
}
```

**Pagination patterns:**

- **Offset pagination** (`?page=2&limit=20`) — simple, supports jumping to any page. Bad for real-time data (inserting/deleting items shifts offsets).
- **Cursor pagination** (`?cursor=abc123&limit=20`) — stable, no skipped/duplicate items. Better for feeds and real-time data. Can't jump to page 50 directly.

**Debouncing API calls:**

JAVASCRIPT Copy

```
function useDebouncedSearch(delay = 300) {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);

  useEffect(() => {
    if (!query) { setResults([]); return; }

    const timer = setTimeout(() => {
      fetch(`/api/search?q=${query}`).then(r => r.json()).then(setResults)
    }, delay);

    return () => clearTimeout(timer);
  }, [query, delay]);

  return { query, setQuery, results };
}
```

**Follow-up:** "How does TanStack Query / React Query help with API optimization?"

It provides automatic caching (don't re-fetch data you already have), deduplication (10 components requesting the same data = 1 API call), background refetching (stale-while-revalidate pattern), and retry with exponential backoff. It turns API state management from a manual problem into a declarative one.

## Q 8.15 How do you build performant animations? Explain `requestAnimationFrame`, FLIP, and View Transitions.

**Difficulty:** Senior **Asked at:** Google, Amazon, Flipkart, Microsoft

Covered in detail in Chapter 3 (Q3.11). Key summary for performance context:

**The rule:** Only animate `transform` and `opacity`. These run on the GPU compositor thread and don't trigger layout or paint. Animating `width`, `height`, `top`, `left`, `margin`, or `padding` triggers layout recalculation on every frame — 60 times per second.

`requestAnimationFrame` **— sync with the display:**

JAVASCRIPT Copy

```
function animate() {
  element.style.transform = `translateX(${position}px)`;
  position += velocity;

  if (position < maxPosition) {
    requestAnimationFrame(animate);  // Next frame
  }
}
requestAnimationFrame(animate);
```

`requestAnimationFrame` fires once per display refresh (60fps = every 16.6ms). It pauses when the tab is hidden (saves CPU/battery). Never use `setInterval` for animations — it's not synced to the display refresh rate and causes frame drops.

**FLIP for layout animations** — see Chapter 3, Q3.11 for the full implementation. In short: record position before DOM change, apply the change, calculate the difference, animate from old position to new using `transform`.

**View Transitions** — see Chapter 3, Q3.11. Browser-native cross-state transitions that capture before/after screenshots and animate between them.

**Follow-up:** "What's a good frame budget?"

At 60fps, each frame has 16.6ms. Your JavaScript should finish in under 10ms to leave time for style calculation, layout, and paint. Use the Performance tab's flame chart to

identify frames that exceed 16.6ms.



## Q 8.16 What is perceived performance? How do skeleton screens and optimistic updates help?

**Difficulty:** Mid **Asked at:** Flipkart, Swiggy, Amazon, Razorpay

Perceived performance is how fast the experience feels to the user, which is often more important than actual load times. A page that shows a skeleton screen in 200ms and fills in content at 1.5s feels faster than one that shows a blank page for 1.2s and then renders everything at once — even though the total time is slightly longer.

**Skeleton screens:**

JSX Copy

```jsx
function ProductCard({ product }) {
  if (!product) {
    return (
      <div className="card">
        <div className="skeleton" style={{ width: '100%', height: 200 }} /
        <div className="skeleton" style={{ width: '70%', height: 20, margi
        <div className="skeleton" style={{ width: '40%', height: 16, margi
      </div>
    );
  }

  return (
    <div className="card">
      <img src={product.image} alt={product.name} />
      <h3>{product.name}</h3>
      <span>₹{product.price}</span>
    </div>
  );
}
```

Skeletons work because they set spatial expectations. The user's brain processes the layout — "there's a card with an image, a title, and a price" — before the actual data

**Optimistic updates:**

Show the result of an action immediately without waiting for the server to confirm:

JSX Copy

```jsx
function LikeButton({ postId, initialCount }) {
  const [liked, setLiked] = useState(false);
  const [count, setCount] = useState(initialCount);

  async function handleLike() {
    // Update UI immediately (optimistic)
    setLiked(true);
    setCount(c => c + 1);

    try {
      await fetch(`/api/posts/${postId}/like`, { method: 'POST' });
    } catch {
      // Rollback if server fails
      setLiked(false);
      setCount(c => c - 1);
    }
  }

  return (
    <button onClick={handleLike}>
      {liked ? '❤️' : '🤍'} {count}
    </button>
  );
}
```

The like counter increments instantly. If the API fails, it rolls back. In practice, the API succeeds 99.9% of the time, so users experience zero latency for likes, follows, and similar actions.

**Progressive loading:**

Load and render content in priority order. On a product page: render the product title and price first (above the fold), then the image, then descriptions, then reviews. Each

When the action has irreversible consequences (sending money, deleting an account) or when the failure rate is high (unreliable network, rate-limited APIs). For these, use a loading state and wait for server confirmation.

---

## Q 8.17 How do you monitor frontend performance in production?

**Difficulty:** Senior **Asked at:** Google, Amazon, Flipkart, Razorpay

**Lab testing vs real-user monitoring (RUM):**

Lab testing (Lighthouse, WebPageTest) measures performance in controlled conditions — consistent network, consistent device, no browser extensions. Good for development and CI, but doesn't represent real users.

RUM measures actual users on real devices and networks. A Lighthouse score of 95 means nothing if real users on budget Android phones with 3G connections have 8-second LCP.

**Measuring in production:**

JAVASCRIPT Copy

```
import { onLCP, onINP, onCLS, onFCP, onTTFB } from 'web-vitals';

function sendMetric(metric) {
  // Send to your analytics/monitoring service
  fetch('/api/vitals', {
    method: 'POST',
    body: JSON.stringify({
      name: metric.name,
      value: metric.value,
      rating: metric.rating,     // "good", "needs-improvement", or "poor"
      navigationType: metric.navigationType,
      url: window.location.href,
      userAgent: navigator.userAgent,
```

```
  });
}

onLCP(sendMetric);
onINP(sendMetric);
onCLS(sendMetric);
onFCP(sendMetric);
onTTFB(sendMetric);
```

**What to track beyond Core Web Vitals:**

- JavaScript errors (Sentry, Datadog) — error rate by page, by browser, by device
- Long tasks — how often the main thread is blocked for >50ms
- API response times — p50, p95, p99 from the frontend perspective
- Resource loading failures — images that 404, scripts that timeout

**Dashboarding and alerting:**

Track Core Web Vitals as percentiles, not averages. The p75 (75th percentile) is what Google uses for ranking. Alert when p75 LCP exceeds 2.5s or p75 INP exceeds 200ms. Segment by device type (mobile vs desktop), connection speed, and geography — mobile users in India have very different performance than desktop users in the US.

**Follow-up:** "What's the difference between Lighthouse CI and RUM?"

Lighthouse CI runs Lighthouse in your build pipeline, giving consistent lab measurements on every PR. It catches regressions before deployment. RUM tells you what's happening after deployment in the real world. You need both — Lighthouse CI as a gate, RUM as the ground truth.

---

## Q 8.18 What are performance budgets? How do you set and enforce them?

**Difficulty:** Senior **Asked at:** Google, Amazon, Razorpay

A performance budget is a hard limit on metrics that affect user experience. "The total JavaScript bundle must be under 200KB gzipped." "LCP must be under 2.5 seconds." "No single third-party script may exceed 50KB."

Without budgets, performance degrades gradually. Each sprint adds a library here, a feature there, and nobody notices the 5KB increments until the page takes 6 seconds to load.

**Types of budgets:**

- **Size budgets:** Total JS < 200KB, total CSS < 50KB, total images < 500KB per page
- **Timing budgets:** LCP < 2.5s, INP < 200ms, TTFB < 600ms
- **Count budgets:** < 50 HTTP requests per page, < 5 third-party scripts

**Setting budgets:**

Start from your current metrics and your competition. If your LCP is 4 seconds and competitors are at 2.5s, set 2.5s as the target. For bundle size, analyze your current bundle, identify the ideal minimum, and set the budget between current and ideal.

**Enforcing in CI/CD:**

JAVASCRIPT Copy

```javascript
// .lighthouserc.js
module.exports = {
  ci: {
    assert: {
      assertions: {
        'categories:performance': ['error', { minScore: 0.9 }],
        'largest-contentful-paint': ['error', { maxNumericValue: 2500 }],
        'interactive': ['error', { maxNumericValue: 3500 }],
        'total-byte-weight': ['error', { maxNumericValue: 500000 }],
      },
    },
  },
};
```

YAML Copy

```
- name: Lighthouse CI
  run: npx @lhci/cli autorun
  # Fails the build if budgets are exceeded
```

**Bundle size monitoring:**

JSON Copy

```
// package.json - using bundlesize
{
  "bundlesize": [
    { "path": "dist/js/*.js", "maxSize": "200 kB" },
    { "path": "dist/css/*.css", "maxSize": "50 kB" }
  ]
}
```

Every PR that increases bundle size beyond the budget fails the check. The developer must either optimize their code, remove something else, or make a case for increasing the budget.

**Follow-up:** "What happens when a budget is exceeded?"

Treat it like a failing test. The PR can't merge until the budget is met. If there's a legitimate reason to exceed it (new feature with unavoidable cost), the budget increase must be explicitly approved and documented. This prevents the slow creep that kills performance over months.

QUICK CHECK — no marks, just for you.

1. Your build pipeline currently runs Brotli over every file it emits, including the .woff2 fonts and .webp images. According to the lesson, what is wrong with this?

- ○ Brotli cannot process binary files at all, so the build step will fail on fonts and images
- ○ Browsers ignore Content-Encoding on font and image responses, so the compression is silently discarded

Those formats are already compressed, so compressing them wastes CPU and may even increase file size

- Brotli only supports text MIME types below 200KB, so larger assets are skipped anyway

Check answer

2. You are paginating a live activity feed where new items are inserted at the top constantly. Per the lesson, why is offset pagination a poor fit here?

- Inserting or deleting items shifts the offsets, so users can see skipped or duplicated items
- Offset pagination responses cannot be cached, so every scroll re-hits the origin server
- Offset pagination prevents jumping directly to an arbitrary page, which feeds require
- Offset pagination cannot be combined with a limit parameter, so page sizes become unpredictable

Check answer

3. Your RUM dashboard reports average LCP and it looks healthy, yet Google's report flags the page. Following the lesson's guidance on production monitoring, what should the dashboard report instead?

- The median (p50), because averages are skewed by fast desktop sessions
- The p75 (75th percentile), because that is the percentile Google uses for ranking
- The Lighthouse lab score, because it is the only measurement Google actually consumes
- The p99, because ranking is determined by the slowest users on the site

Previous
Runtime and network

Next
Creational patterns

## LEARN MORE

### ARTICLES & DOCS

Web Vitals (web.dev)
web.dev

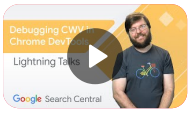
### VIDEOS




Understanding performance with Core Web Vitals
YouTube



Check Core Web Vitals with Lighthouse
YouTube




Debugging Core Web Vitals with DevTools
YouTube

Hand-picked references for this lesson.